

SBA301

DỰ ÁN THỰC HÀNH Củng Cố SLOT 02

SBA301 Student Learning Dashboard

Hướng dẫn chi tiết từng bước xây dựng dự án ReactJS mức độ trung bình với IntelliJ IDEA

Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Phạm vi củng cố	Slot 02 – ReactJS Fundamentals, Vite, JSX, Components, Import/Export, Rendering, Git
Mức độ	Trung bình – nhiều component, module dữ liệu, JSX expression, composition, CSS, Git
IDE	IntelliJ IDEA
Frontend	ReactJS + Vite + JavaScript + JSX
Nguyên tắc	Không dùng Props/State/Hooks/Router trong core project

Mục tiêu của tài liệu — Sinh viên có thể tự tạo project, cấu hình IntelliJ, viết và tổ chức React components, chạy/kiểm tra/debug ứng dụng, lưu Git checkpoint và chứng minh kết quả bằng evidence.

1. Tóm tắt dự án

Dự án “SBA301 Student Learning Dashboard” là một dashboard ReactJS tĩnh nhưng được tổ chức theo kiến trúc component rõ ràng. Dữ liệu học tập, thông tin sinh viên, trạng thái môi trường và project group được đặt trong module JavaScript riêng; các functional components đọc dữ liệu và render bằng JSX. Độ khó trung bình đến từ việc phải tổ chức nhiều file, hiểu luồng import/export, component tree, JSX expressions, array rendering, CSS và quy trình chạy/debug/Git — nhưng vẫn không đi trước sang Props, State, Hooks hoặc Router.

1.1. Mục tiêu học tập

- ☐ Giải thích được React component là gì và vì sao UI nên chia thành các phần nhỏ có trách nhiệm rõ ràng.
- ☐ Tạo và chạy project React bằng Vite trong IntelliJ IDEA.
- ☐ Đọc được luồng index.html → main.jsx → App.jsx → child components → browser.
- ☐ Sử dụng JSX, Fragment, JavaScript expressions và className đúng cú pháp.
- ☐ Tạo functional component, đặt tên chữ hoa, export/import đúng đường dẫn.
- ☐ Tổ chức component tree và ghép nhiều component thành một trang hoàn chỉnh.
- ☐ Tách dữ liệu tĩnh sang JavaScript module và render object/array bằng JSX.
- ☐ Biết dùng IntelliJ Terminal, npm scripts, browser console và terminal output để debug.
- ☐ Tạo Git checkpoint sau một trạng thái chạy ổn định.
- ☐ Tự kiểm chứng sản phẩm bằng checklist và evidence thay vì chỉ nhìn giao diện.

1.2. Các chức năng của ứng dụng

#	Chức năng	Mô tả
1	Course Header	Hiển thị mã môn, tên môn, slot và chủ đề hiện tại.
2	Student Profile	Hiển thị tên, mã sinh viên, nhóm và vai trò học tập.
3	Learning Checklist	Hiển thị các mục tiêu Slot 02 dưới dạng danh sách từ JavaScript array.
4	Environment Status	Hiển thị trạng thái Node.js, npm, IntelliJ, Vite và Git bằng badge.
5	Project Summary	Hiển thị tên project nhóm, target users và core features.
6	Architecture Flow	Mô tả mental model main.jsx → App → Components → JSX → Browser.
7	Footer	Hiển thị thông tin môn học và thông điệp verification.

1.3. Công nghệ sử dụng

Công nghệ/Công cụ	Vai trò
ReactJS	UI library; functional components + JSX
Vite 5.x	Scaffold/dev server/build tool trong môi trường Node 21 của

	syllabus
JavaScript	Dữ liệu, module, expression và array processing
CSS	Trang trí và responsive cơ bản
IntelliJ IDEA	IDE chính để tạo, chỉnh sửa, chạy và debug project
Node.js + npm	Runtime/tooling và package manager
Git	Version control và working checkpoint
Chrome/Edge DevTools	Kiểm tra Console và DOM

Lưu ý phiên bản — Syllabus khóa Node 21. Vite hiện hành yêu cầu các dòng Node mới hơn; vì vậy tài liệu này dùng create-vite 5.x cho đường học chính. Nếu giảng viên đã nâng Node lên phiên bản được Vite hiện hành hỗ trợ, có thể chuyển sang @latest theo thông báo chính thức của lớp.

2. Cấu trúc thư mục mục tiêu

```

sba301-learning-dashboard/
├── public/
├── src/
│   ├── components/
│   │   ├── ArchitectureFlow.jsx
│   │   ├── CourseHeader.jsx
│   │   ├── EnvironmentStatus.jsx
│   │   ├── LearningChecklist.jsx
│   │   ├── ProjectSummary.jsx
│   │   ├── StudentProfile.jsx
│   │   └── Footer.jsx
│   ├── data/
│   │   └── dashboardData.js
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── index.html
├── package.json
└── vite.config.js

```

Luồng render cần nhớ:

```

Browser
  ↓
index.html (#root)
  ↓
main.jsx (createRoot + render)
  ↓
App.jsx
  ↓
CourseHeader / StudentProfile / LearningChecklist / ...
  ↓
JSX
  ↓
Rendered UI

```

3. Tóm tắt các bước thực hiện

Mã	Bước	Nội dung	Checkpoint
B0	Kiểm tra điều kiện trước khi bắt đầu	Node/npm, IntelliJ, Git	PASS ☐ / FAIL ☐
B1	Cấu hình IntelliJ cho JavaScript/Node/Vite	Plugin + Node interpreter + Terminal	PASS ☐ / FAIL ☐
B2	Tạo thư mục workspace	SBA301/labs	PASS ☐ / FAIL ☐
B3	Scaffold React project bằng Vite 5	npm create vite@5 ...	PASS ☐ / FAIL ☐
B4	Mở project và cài dependencies	npm install	PASS ☐ / FAIL ☐
B5	Chạy baseline mặc định	npm run dev	PASS ☐ / FAIL ☐
B6	Đọc cấu trúc project	index.html, main.jsx, App.jsx, package.json	PASS ☐ / FAIL ☐
B7	Làm sạch starter template	Giữ baseline tối giản	PASS ☐ / FAIL ☐
B8	Tạo module dữ liệu	dashboardData.js	PASS ☐ / FAIL ☐
B9	Tạo 7 functional components	Component + export/import	PASS ☐ / FAIL ☐
B10	Ghép component trong App.jsx	Component composition	PASS ☐ / FAIL ☐
B11	Tạo CSS cho dashboard	index.css	PASS ☐ / FAIL ☐
B12	Chạy và kiểm tra từng vùng UI	Browser + Console	PASS ☐ / FAIL ☐
B13	Cấu hình Run Configuration trong IntelliJ	npm dev	PASS ☐ / FAIL ☐
B14	Thực hiện mutation test đơn giản	Sửa data → quan sát UI	PASS ☐ / FAIL ☐
B15	Tạo Git checkpoint	status/add/commit/log	PASS ☐ / FAIL ☐
B16	Build production và preview	npm run build / preview	PASS ☐ / FAIL ☐
B17	Hoàn thành Final Verification Gate	Evidence + oral explanation	PASS ☐ / FAIL ☐

4. Hướng dẫn chi tiết từng bước

Bước 0 — Kiểm tra điều kiện trước khi bắt đầu

Mở PowerShell hoặc Terminal của IntelliJ và chạy lần lượt:

```
node -v
npm -v
git --version
```

Trong môi trường syllabus chuẩn, Node.js được khóa ở dòng 21. Nếu máy dùng Node khác, không tự ý đổi nếu lớp đang yêu cầu đồng nhất môi trường; ghi lại version để giảng viên kiểm tra.

Kết quả mong đợi	Cả ba lệnh chạy được; không có thông báo “not recognized / command not found”.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	Ảnh/chụp hoặc ghi text của node -v, npm -v, git --version

Bước 1 — Cấu hình IntelliJ IDEA

1.1. Kiểm tra plugin JavaScript/TypeScript và Vite

1. Mở IntelliJ IDEA.
2. Chọn File → Settings (Windows/Linux) hoặc IntelliJ IDEA → Settings/Preferences (macOS).
3. Chọn Plugins → Installed.
4. Tìm “JavaScript and TypeScript”. Đảm bảo plugin ở trạng thái Enabled.
5. Tìm “Vite”. Đảm bảo plugin ở trạng thái Enabled.
6. Nếu vừa bật plugin, restart IDE khi IntelliJ yêu cầu.

Kết quả mong đợi	IntelliJ nhận diện file .js/.jsx, syntax highlighting hoạt động và Vite support được bật.
Đánh giá	PASS ☐ FAIL ☐

1.2. Cấu hình Node.js runtime trong IntelliJ

7. File → Settings → Languages & Frameworks → Node.js (tên menu có thể thay đổi nhẹ theo phiên bản).
8. Tại Node interpreter/runtime, chọn Node đã cài trên máy.
9. Nếu danh sách trống, nhấn Add... và trở tới executable node (Windows thường nằm trong C:\Program Files\nodejs\node.exe).
10. Kiểm tra npm package manager được IntelliJ nhận diện.
11. Nhấn Apply → OK.

Kết quả mong đợi	Node interpreter xuất hiện trong IntelliJ; Terminal chạy node -v và npm -v thành công.
Đánh giá	PASS ☐ FAIL ☐

Bước 2 — Tạo workspace cho môn học

Trong IntelliJ, mở Terminal bằng Alt+F12 (hoặc View → Tool Windows → Terminal). Chuyển tới thư mục học tập và tạo labs nếu chưa có.

```
mkdir SBA301
cd SBA301
mkdir labs
cd labs
```

Windows PowerShell — Nếu thư mục đã tồn tại, bỏ qua lệnh mkdir tương ứng. Dùng Get-Location để kiểm tra vị trí hiện tại.

Kết quả mong đợi	Terminal hiện đang ở thư mục .../SBA301/labs.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	Kết quả Get-Location hoặc pwd

Bước 3 — Tạo project React bằng Vite

Để phù hợp môi trường Node 21 của syllabus, dùng create-vite 5.x thay vì @latest:

```
npm create vite@5 sba301-learning-dashboard -- --template react
```

Ý nghĩa:

npm: package manager

create vite@5: chạy create-vite thuộc major version 5

sba301-learning-dashboard: tên thư mục project

--template react: chọn template React + JavaScript

Sau khi command kết thúc, kiểm tra có thư mục sba301-learning-dashboard được tạo.

Kết quả mong đợi	Thư mục project tồn tại và chứa package.json, src, index.html, vite.config.*.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	Tree/File view của IntelliJ hoặc dir/ls

Bước 4 — Mở project trong IntelliJ và cài dependency

- Trong IntelliJ chọn File → Open.
- Chọn thư mục sba301-learning-dashboard.
- Nếu IntelliJ hỏi Trust Project, chọn Trust Project vì đây là project do chính bạn tạo.
- Chờ IDE indexing hoàn tất.
- Mở Terminal trong project và kiểm tra vị trí hiện tại phải kết thúc bằng sba301-learning-dashboard.
- Chạy npm install.

```
npm install
```

Sau npm install, npm đọc package.json và tải dependencies vào node_modules. Không chỉnh sửa thủ công node_modules.

Kết quả mong đợi	node_modules xuất hiện; npm install kết thúc không có error blocking.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	Terminal output + thư mục node_modules

Bước 5 — Chạy baseline React trước khi sửa code

```
npm run dev
```

Terminal sẽ in Local URL, thông thường dạng `http://localhost:5173/`. Ctrl+Click vào URL hoặc mở browser và nhập URL đó.

- ☐ Trang starter React/Vite hiển thị.
- ☐ Terminal không có compile error.
- ☐ F12 → Console không có lỗi ứng dụng nghiêm trọng.

Quy tắc baseline — Không sửa code cho tới khi starter project chạy được. Nếu baseline không chạy, lỗi thuộc setup/tooling nhiều khả năng hơn lỗi code của bạn.

Kết quả mong đợi	Starter page tải được qua browser; dev server vẫn đang chạy.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	URL + ảnh giao diện + Console

Bước 6 — Đọc cấu trúc project trước khi sửa

Trong Project panel của IntelliJ, mở lần lượt:

index.html — browser entry document; có div id="root"

src/main.jsx — tạo React root và render App

src/App.jsx — root application component của starter

src/index.css — CSS global

package.json — scripts + dependencies + devDependencies

vite.config.js — cấu hình Vite

```

index.html
  ↓
main.jsx
  ↓
<App />
  ↓
App.jsx
  ↓
Browser UI

```

Kết quả mong đợi	Sinh viên chỉ đúng được file entry, root mounting point và App component mà không cần chạy tìm kiếm.
------------------	--

Đánh giá	PASS ☐ FAIL ☐
----------	---

Bước 7 — Làm sạch starter template

Dừng dev server bằng Ctrl+C nếu muốn chỉnh sửa yên tĩnh, hoặc giữ server chạy để quan sát hot reload. Mở src/App.jsx và thay nội dung bằng baseline tối giản:

src/App.jsx

```
function App() {
  return (
    <main>
      <h1>SBA301 Student Learning Dashboard</h1>
    </main>
  );
}

export default App;
```

Mở src/index.css và xóa CSS starter phức tạp; tạm thay bằng:

src/index.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #f4f6f8;
  color: #1f2937;
}
```

Kết quả mong đợi	Browser chỉ hiển thị tiêu đề dashboard, không có logo/counter starter và Console sạch.
Đánh giá	PASS ☐ FAIL ☐

Bước 8 — Tạo module dữ liệu dùng chung

Trong src, tạo folder data. IntelliJ: right-click src → New → Directory → data. Sau đó tạo file dashboardData.js.

src/data/dashboardData.js

```
export const course = {
  code: "SBA301",
  name: "Integrate Single Page Application with Spring Boot",
  slot: "Slot 02",
  topic: "ReactJS Fundamentals"
};

export const student = {
  name: "YOUR NAME",
  studentId: "YOUR ID",
  group: "YOUR GROUP",
  role: "Learner / Developer"
};
```

```

export const learningItems = [
  "React mental model",
  "Vite project workflow",
  "JSX and JavaScript expressions",
  "Functional components",
  "Import and export",
  "Component composition",
  "Browser and console verification",
  "Git checkpoint"
];

export const environmentTools = [
  { name: "Node.js", status: "Ready" },
  { name: "npm", status: "Ready" },
  { name: "IntelliJ IDEA", status: "Ready" },
  { name: "Vite", status: "Ready" },
  { name: "Git", status: "Ready" }
];

export const groupProject = {
  name: "YOUR GROUP PROJECT",
  targetUsers: "Define your target users",
  coreFeatures: [
    "Feature 1",
    "Feature 2",
    "Feature 3"
  ]
};

```

Slot 02 extension — Việc tách data sang module riêng là bài mở rộng vừa phải từ import/export. Chúng ta chưa dùng Props hoặc State; component sẽ import dữ liệu trực tiếp.

Kết quả mong đợi	dashboardData.js không báo syntax error; các export được IntelliJ nhận diện.
Đánh giá	PASS ☐ FAIL ☐

Bước 9 — Tạo các functional components

Tạo folder src/components. Mỗi component cần: tên chữ hoa, return JSX hợp lệ, export default. Thực hiện từng component và kiểm tra ngay sau khi import vào App ở Bước 10.

9.1. CourseHeader

Tạo file src/components/CourseHeader.jsx, sau đó nhập code sau:

src/components/CourseHeader.jsx

```

import { course } from "../data/dashboardData";

function CourseHeader() {
  return (
    <header className="hero">
      <p className="eyebrow">{course.slot} • {course.topic}</p>
      <h1>{course.code}</h1>
      <p className="course-name">{course.name}</p>
    </header>
  );
}

```

```

    </header>
  );
}

export default CourseHeader;

```

Kết quả mong đợi	CourseHeader.jsx không có syntax error; tên component bắt đầu bằng chữ hoa; có export default.
Đánh giá	PASS ☐ FAIL ☐

9.2. StudentProfile

Tạo file src/components/StudentProfile.jsx, sau đó nhập code sau:

src/components/StudentProfile.jsx

```

import { student } from "../data/dashboardData";

function StudentProfile() {
  return (
    <section className="card">
      <h2>Student Profile</h2>
      <p><strong>Name:</strong> {student.name}</p>
      <p><strong>Student ID:</strong> {student.studentId}</p>
      <p><strong>Group:</strong> {student.group}</p>
      <p><strong>Role:</strong> {student.role}</p>
    </section>
  );
}

export default StudentProfile;

```

Kết quả mong đợi	StudentProfile.jsx không có syntax error; tên component bắt đầu bằng chữ hoa; có export default.
Đánh giá	PASS ☐ FAIL ☐

9.3. LearningChecklist

Tạo file src/components/LearningChecklist.jsx, sau đó nhập code sau:

src/components/LearningChecklist.jsx

```

import { learningItems } from "../data/dashboardData";

function LearningChecklist() {
  return (
    <section className="card">
      <h2>Slot 02 Learning Checklist</h2>
      <ul className="check-list">
        {learningItems.map((item) => (
          <li key={item}>{item}</li>
        ))}
      </ul>
    </section>
  );
}

```

```
export default LearningChecklist;
```

Kết quả mong đợi	LearningChecklist.jsx không có syntax error; tên component bắt đầu bằng chữ hoa; có export default.
Đánh giá	PASS ☐ FAIL ☐

9.4. EnvironmentStatus

Tạo file src/components/EnvironmentStatus.jsx, sau đó nhập code sau:

src/components/EnvironmentStatus.jsx

```
import { environmentTools } from "../data/dashboardData";

function EnvironmentStatus() {
  return (
    <section className="card">
      <h2>Environment Status</h2>
      <div className="status-list">
        {environmentTools.map((tool) => {
          const badgeClass = tool.status === "Ready" ? "badge ready" : "badge pending";

          return (
            <div className="status-row" key={tool.name}>
              <span>{tool.name}</span>
              <span className={badgeClass}>{tool.status}</span>
            </div>
          );
        })}
      </div>
    </section>
  );
}

export default EnvironmentStatus;
```

Kết quả mong đợi	EnvironmentStatus.jsx không có syntax error; tên component bắt đầu bằng chữ hoa; có export default.
Đánh giá	PASS ☐ FAIL ☐

9.5. ProjectSummary

Tạo file src/components/ProjectSummary.jsx, sau đó nhập code sau:

src/components/ProjectSummary.jsx

```
import { groupProject } from "../data/dashboardData";

function ProjectSummary() {
  return (
    <section className="card">
      <h2>Group Project Summary</h2>
      <p><strong>Project:</strong> {groupProject.name}</p>
      <p><strong>Target users:</strong> {groupProject.targetUsers}</p>
      <h3>Core Features</h3>
    </section>
  );
}
```

```

    <ol>
      {groupProject.coreFeatures.map((feature) => (
        <li key={feature}>{feature}</li>
      ))}
    </ol>
  </section>
);
}

export default ProjectSummary;

```

Kết quả mong đợi	ProjectSummary.jsx không có syntax error; tên component bắt đầu bằng chữ hoa; có export default.
Đánh giá	PASS ☐ FAIL ☐

9.6. ArchitectureFlow

Tạo file src/components/ArchitectureFlow.jsx, sau đó nhập code sau:

src/components/ArchitectureFlow.jsx

```

function ArchitectureFlow() {
  return (
    <section className="card architecture-card">
      <h2>React Rendering Mental Model</h2>
      <div className="flow">
        <span>index.html</span>
        <span>→</span>
        <span>main.jsx</span>
        <span>→</span>
        <span>App</span>
        <span>→</span>
        <span>Components</span>
        <span>→</span>
        <span>JSX</span>
        <span>→</span>
        <span>Browser</span>
      </div>
    </section>
  );
}

export default ArchitectureFlow;

```

Kết quả mong đợi	ArchitectureFlow.jsx không có syntax error; tên component bắt đầu bằng chữ hoa; có export default.
Đánh giá	PASS ☐ FAIL ☐

9.7. Footer

Tạo file src/components/Footer.jsx, sau đó nhập code sau:

src/components/Footer.jsx

```

function Footer() {
  const year = new Date().getFullYear();

```

```

    return (
      <footer className="footer">
        <p>SBA301 • {year} • Build → Run → Verify → Explain</p>
      </footer>
    );
  }

  export default Footer;

```

Kết quả mong đợi	Footer.jsx không có syntax error; tên component bắt đầu bằng chữ hoa; có export default.
Đánh giá	PASS ☐ FAIL ☐

Giải thích .map() — `learningItems.map(...)` và `environmentTools.map(...)` là thao tác JavaScript trên array để tạo nhiều JSX elements. Đây không phải State, Hook hay dữ liệu backend. Dùng ở đây để nâng độ khó dự án mà vẫn nằm gần nền tảng JavaScript/JSX.

Bước 10 — Ghép component trong App.jsx

Mở `src/App.jsx`, import tất cả component rồi compose UI. Đây là điểm chính để củng cố component composition.

`src/App.jsx`

```

import ArchitectureFlow from "../components/ArchitectureFlow";
import CourseHeader from "../components/CourseHeader";
import EnvironmentStatus from "../components/EnvironmentStatus";
import Footer from "../components/Footer";
import LearningChecklist from "../components/LearningChecklist";
import ProjectSummary from "../components/ProjectSummary";
import StudentProfile from "../components/StudentProfile";

function App() {
  return (
    <>
      <CourseHeader />

      <main className="dashboard">
        <StudentProfile />
        <EnvironmentStatus />
        <LearningChecklist />
        <ProjectSummary />
        <ArchitectureFlow />
      </main>

      <Footer />
    </>
  );
}

export default App;

```

Component tree hiện tại:

App

- CourseHeader
- StudentProfile
- EnvironmentStatus
- LearningChecklist
- ProjectSummary
- ArchitectureFlow
- Footer

Kết quả mong đợi	App.jsx compile; browser render đầy đủ các section; không có “X is not defined” hoặc import error.
Đánh giá	PASS ☐ FAIL ☐

Bước 11 — Tạo CSS hoàn chỉnh cho dashboard

Thay toàn bộ src/index.css bằng CSS sau. CSS không phải trọng tâm React, nhưng giúp dự án có sản phẩm rõ ràng để kiểm tra component composition.

src/index.css

```
* {
  box-sizing: border-box;
}

:root {
  font-family: Inter, Arial, Helvetica, sans-serif;
  color: #1f2937;
  background: #eef2f7;
}

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
}

.hero {
  padding: 32px 24px;
  text-align: center;
  background: #17365d;
  color: white;
}

.hero h1 {
  margin: 4px 0;
  font-size: 42px;
}

.eyebrow {
  margin: 0;
  font-size: 14px;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}

.course-name {
  margin: 8px auto 0;
  max-width: 760px;
}
```

```

}

.dashboard {
  width: min(1100px, 92%);
  margin: 28px auto;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 18px;
}

.card {
  background: white;
  border: 1px solid #dbe3ee;
  border-radius: 12px;
  padding: 20px;
  box-shadow: 0 4px 14px rgba(15, 23, 42, 0.06);
}

.card h2 {
  margin-top: 0;
  color: #17365d;
}

.card h3 {
  color: #2f5597;
}

.check-list {
  margin: 0;
  padding-left: 22px;
}

.check-list li {
  margin: 8px 0;
}

.status-list {
  display: grid;
  gap: 10px;
}

.status-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 0;
  border-bottom: 1px solid #edf0f4;
}

.badge {
  padding: 4px 10px;
  border-radius: 999px;
  font-size: 13px;
  font-weight: 700;
}

.ready {
  background: #dcfce7;
}

```

```

    color: #166534;
  }

  .pending {
    background: #fef3c7;
    color: #92400e;
  }

  .architecture-card {
    grid-column: 1 / -1;
  }

  .flow {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
    align-items: center;
  }

  .flow span:nth-child(odd) {
    background: #eaf2fb;
    border: 1px solid #b8d0ea;
    border-radius: 8px;
    padding: 8px 12px;
    font-weight: 600;
  }

  .footer {
    padding: 22px;
    text-align: center;
    color: #475569;
  }

  @media (max-width: 760px) {
    .dashboard {
      grid-template-columns: 1fr;
    }

    .architecture-card {
      grid-column: auto;
    }
  }

```

Kết quả mong đợi	Dashboard có header xanh đậm, các card 2 cột trên màn hình rộng và 1 cột trên màn hình hẹp; không có CSS parsing error.
Đánh giá	PASS ☐ FAIL ☐

Bước 12 — Chạy và kiểm tra ứng dụng

Nếu dev server đã dừng, chạy lại:

```
npm run dev
```

Kiểm tra theo thứ tự sau:

- ☐ URL local mở được.
- ☐ CourseHeader hiển thị SBA301 / Slot 02 / ReactJS Fundamentals.

- ☐ StudentProfile hiển thị dữ liệu từ dashboardData.js.
- ☐ LearningChecklist hiển thị đủ danh sách.
- ☐ EnvironmentStatus hiển thị badge Ready.
- ☐ ProjectSummary hiển thị 3 core features.
- ☐ ArchitectureFlow hiển thị đủ chuỗi index.html → main.jsx → App → Components → JSX → Browser.
- ☐ F12 → Console không có relevant error.
- ☐ Terminal không có compile error.

Kết quả mong đợi	Tất cả 9 kiểm tra đạt PASS.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	Screenshot toàn trang + Console + terminal dev server

Bước 13 — Tạo Run Configuration trong IntelliJ

Mục tiêu: chạy project bằng nút Run thay vì luôn gõ command.

18. Chọn Run → Edit Configurations...
19. Nhấn + → npm.
20. Name: React Dev Server.
21. package.json: chọn package.json của sba301-learning-dashboard.
22. Command: run.
23. Scripts: dev.
24. Node interpreter: chọn interpreter đã cấu hình ở Bước 1.
25. Nhấn Apply → OK.
26. Chọn configuration “React Dev Server” trên toolbar và nhấn Run.

Kết quả mong đợi	Run tool window hiển thị Vite dev server; URL local xuất hiện; browser truy cập được.
Đánh giá	PASS ☐ FAIL ☐

Bước 14 — Mutation test để chứng minh data → JSX → UI

Không dùng AI. Mở src/data/dashboardData.js, đổi:

```
student: đổi name / studentId
groupProject.name: đổi sang tên project thật của nhóm
environmentTools: tạm đổi Git từ "Ready" thành "Pending"
```

Lưu file và quan sát browser. Vite hot update phải cập nhật giao diện mà không cần restart toàn bộ project. Badge Git phải đổi màu do biểu thức className.

- ☐ Giải thích được giá trị nào thay đổi trong data file.
- ☐ Chỉ ra component nào đọc dữ liệu đó.
- ☐ Chỉ ra JSX expression nào render dữ liệu.
- ☐ Chỉ ra bằng chứng browser đã cập nhật.

Kết quả mong đợi	Sinh viên chứng minh được data → component → JSX → browser, không gọi đây là State.
Đánh giá	PASS ☐ FAIL ☐

Bước 15 — Git checkpoint

Mở IntelliJ Terminal tại project root:

```
git status
```

Nếu project chưa là Git repository:

```
git init
```

Sau đó:

```
git add .
git status
git commit -m "Build Slot 02 React learning dashboard"
git log -1
```

Phân biệt — git commit tạo checkpoint local. Nó không đồng nghĩa code đã được push lên GitHub/GitLab.

Kết quả mong đợi	git log -1 hiển thị commit vừa tạo; working tree sạch hoặc chỉ còn file bạn chủ động chưa commit.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	Commit hash + message

Bước 16 — Build production và preview

Đây là bước kiểm tra bổ sung để chắc project không chỉ chạy ở dev mode.

```
npm run build
```

Kết quả thành công sẽ tạo thư mục dist. Sau đó preview build:

```
npm run preview
```

Mở URL do preview command cung cấp. Dừng preview bằng Ctrl+C.

Kết quả mong đợi	npm run build kết thúc thành công; dist được tạo; preview tải được giao diện.
Đánh giá	PASS ☐ FAIL ☐
Evidence cần ghi	Build output + dist + preview URL

Bước 17 — Final Verification Gate

Tiêu chí	Cách kiểm tra	Evidence	Kết quả
Project scaffold	package.json + src + index.html	Project tree	PASS/FAIL

Tiêu chí	Cách kiểm tra	Evidence	Kết quả
Dev server	npm run dev	Terminal URL	PASS/FAIL
Component tree	App imports 7 components	App.jsx	PASS/FAIL
JSX expression	{course.code}, {student.name}, map()	Code + UI	PASS/FAIL
Capitalized component	CourseHeader, StudentProfile...	Component files	PASS/FAIL
Import/export	All child components imported/exported	No module error	PASS/FAIL
Fragment	App wraps output with <>...</>	App.jsx	PASS/FAIL
Browser verification	All cards visible	Screenshot	PASS/FAIL
Console	No relevant error	F12 Console	PASS/FAIL
Git	Meaningful commit	git log -1	PASS/FAIL
Production build	npm run build	dist created	PASS/FAIL
Explanation	Student traces main.jsx → App → child → JSX → browser	Oral 60 seconds	PASS/FAIL

5. Debugging và lỗi thường gặp

5.1. Debugging decision tree

```

Ứng dụng không đúng
↓
npm run dev có chạy không?
↓
Browser có mở đúng Local URL không?
↓
Terminal có compile error không?
↓
Browser Console có error không?
↓
Import path đúng không?
↓
Component có export default không?
↓
Tên component có viết hoa không?
↓
JSX có hợp lệ không?
↓
Data import có đúng export name không?
↓
CSS chỉ là lỗi hiển thị hay code không render?

```

Triệu chứng	Nguyên nhân khả dĩ	Evidence đầu tiên	Hành động tối thiểu
node/npm not recognized	Node chưa cài hoặc PATH/runtime sai	node -v / npm -v	Sửa Node interpreter/PATH rồi kiểm tra lại

Triệu chứng	Nguyên nhân khả dĩ	Evidence đầu tiên	Hành động tối thiểu
npm create vite@latest báo unsupported Node	Latest Vite không hợp Node 21 syllabus	đọc error version	Dùng create-vite@5 theo hướng dẫn lớp
npm run dev báo missing script	Sai folder hoặc package.json không phải project Vite	Get-Location + mở package.json	cd đúng project root
Failed to resolve import	Sai relative path / sai tên file	terminal error chỉ file/line	so đường dẫn từ file đang import
X is not defined	Thiếu import hoặc sai tên component/data	terminal/console	kiểm tra import + export
Element/component lowercase	Tên component user-defined viết thường	JSX code	đổi CourseHeader thay vì courseHeader
Blank page	Compile/runtime/JSX error	terminal + Console	đọc lỗi đầu tiên, không xóa project
map is not a function	Dữ liệu không phải array	console + dashboardData	kiểm tra learningItems/environmentTools
Duplicate key warning	key không unique	Console warning	dùng key ổn định/unique
CSS không áp dụng	index.css chưa import hoặc selector sai	main.jsx + DevTools	kiểm tra import ./index.css
Port 5173 bận	port đang được process khác dùng	terminal	dùng URL port Vite tự chọn hoặc dừng process cũ
Commit không được	Git user chưa cấu hình	git error	cấu hình user.name/user.email theo quy định lớp

5.2. Mẫu báo lỗi trước khi hỏi giảng viên/AI

```

Expected:
[Điều bạn mong đợi]

Actual:
[Điều thực tế xảy ra]

Command:
[npm run dev / npm install / ...]

Terminal error:
[Dòng lỗi đầu tiên có ý nghĩa]

Browser console:
[Lỗi console nếu có]

Relevant file + line:
[App.jsx line ...]

What I already checked:
[import path / export / JSX / current directory / ...]

```

6. Kiểm tra kiến thức sau dự án

1. Vite có phải React không? Giải thích vai trò khác nhau.
2. index.html, main.jsx và App.jsx liên hệ với nhau thế nào?
3. Tại sao CourseHeader phải bắt đầu bằng chữ hoa?
4. JSX khác HTML ở những điểm nào bạn đã sử dụng trong dự án?
5. Dấu {} trong JSX dùng làm gì? Cho 3 ví dụ từ project.
6. Tại sao App.jsx không cần chứa toàn bộ UI chi tiết?
7. import và export trong project này giải quyết vấn đề gì?
8. learningItems.map(...) tạo ra điều gì?
9. Tại sao việc đổi dữ liệu trong dashboardData.js làm UI thay đổi dù chưa học State?
10. Tại sao đây vẫn là dữ liệu tĩnh source-code chứ không phải dynamic application state?
11. Terminal và Browser Console cung cấp hai loại evidence khác nhau thế nào?
12. Tại sao phải tạo Git commit sau khi project đạt trạng thái chạy tốt?
13. Khi trang trắng, thứ tự debug hợp lý là gì?
14. Mô tả component tree của project không nhìn code.
15. Trace 60 giây: main.jsx → App → EnvironmentStatus → JSX → Browser.

7. Bài mở rộng sau khi hoàn thành core project

Giới hạn — Các mở rộng dưới đây chỉ dùng khi core project đã PASS. Không đưa Props/State/Hooks/Router vào nếu lớp chưa học.

- E1** — Tạo src/data/course.js và src/data/student.js riêng để luyện module import/export.
- E2** — Thêm component CourseResources hiển thị 3 tài nguyên học tập từ array.
- E3** — Đổi một tool thành Pending và giải thích ternary expression chọn CSS class.
- E4** — Thêm một component DebugEvidence hiển thị checklist “Terminal checked / Console checked / Git committed”.
- E5** — Thay tên project nhóm thật, target users và 3 core features từ Session 01.
- E6** — Thực hiện responsive check bằng Chrome Device Toolbar và ghi evidence.

8. Gói evidence sinh viên cần nộp/giữ

- ☐ Screenshot Project tree trong IntelliJ.
- ☐ Ảnh terminal chạy npm run dev và Local URL.
- ☐ Ảnh giao diện dashboard hoàn chỉnh.
- ☐ Ảnh F12 Console không có relevant error.
- ☐ Code App.jsx và ít nhất 2 child components.
- ☐ Code dashboardData.js.

- ☐ Kết quả mutation test: trước/sau thay đổi một data value.
- ☐ git log -1 gồm commit hash + message.
- ☐ Kết quả npm run build.
- ☐ Bảng Final Verification Gate đã điền PASS/FAIL.
- ☐ Giải thích miệng 60 giây luồng main.jsx → App → component → JSX → browser.

9. Cơ sở bài học và phạm vi nguồn

Dự án được thiết kế để củng cố Slot 02 và các khái niệm component trong tài liệu môn học: React component là building block; component trả về React elements; UI có thể chia thành các phần độc lập/tái sử dụng; JSX là cú pháp mở rộng JavaScript; component user-defined phải viết hoa; functional component và component composition là trọng tâm. Class component, lifecycle, Hooks/HOC có xuất hiện trong Chapter 02 nhưng KHÔNG được dùng trong core project này để tránh đi trước phạm vi Slot 02.

Nguồn kỹ thuật kiểm tra cấu hình/version:

- React official documentation – Quick Start / Your First Component.
- Vite official documentation – Getting Started; Vite 5 archived documentation for Node 21-compatible course path.
- JetBrains IntelliJ IDEA documentation – Vite / JavaScript / New Project wizard.
- SBA301 Slot 02 lesson baseline – ReactJS Fundamentals, Vite, JSX & First React Application.
- SBA301 Chapter 02 – React Component.

PHỤ LỤC A — Quick Command Sheet

Lệnh	Mục đích
node -v	Kiểm tra Node
npm -v	Kiểm tra npm
git --version	Kiểm tra Git
npm create vite@5 sba301-learning-dashboard -- --template react	Scaffold project phù hợp Node 21 course path
npm install	Cài dependency
npm run dev	Chạy dev server
npm run build	Build production
npm run preview	Preview production build
git status	Xem thay đổi
git init	Khởi tạo repo nếu cần
git add .	Stage files
git commit -m "..."	Tạo checkpoint
git log -1	Kiểm tra commit cuối

PHỤ LỤC B — Student Result Record

Thông tin	Kết quả sinh viên
Student name	
Student ID	
Class/Group	
Node version	
npm version	
IntelliJ version	
Vite project created	
Dev URL	
Components created	
Console error count	
Git commit hash	
Build result	
Final status	

END OF GUIDE